API Documentation

Coursework Submission Supporting Document

Module	XJCO3011 Web Services and Web Data	Project	ShelfSense API
Document Type	API Documentation	**Version**	3.0.0
Base URL	http://127.0.0.1:8000	**Authentication**	Bearer Token

All endpoint paths below are relative to the base URL shown above.

Table of Contents

1. General Information
2. Authentication
3. System Health
4. Book Discovery
5. Reading Lists
6. Reviews
7. Analytics and Recommendations
8. Endpoint Summary

1. General Information

1.1 Overview

ShelfSense API is a RESTful web API for book discovery, reading list management, review storage, user preference analysis, and recommendation generation. It combines a public external data source, Open Library, with a local SQL-backed application database.

1.2 Authentication Rules

Protected endpoints require a bearer token obtained from:

- POST /auth/login

The token must be included in the request header:

```
Authorization: Bearer <token>
```

Protected endpoints include:

- reading list creation, update, and deletion
- reading list item creation, update, and deletion
- review creation, update, and deletion
- current user lookup

1.3 Data Sources

The API uses two data sources:

- Open Library for book search and book detail lookup

- local SQLite storage for users, cached books, reading lists, and reviews

1.4 Error Format

The API uses a simple error structure centred on the `detail` field.

Examples:

```
{
  "detail": "Authentication required"
}

{
  "detail": "Invalid email format"
}
```

For some validation cases, multiple issues may be returned as a list:

```
{
  "detail": [
    "Missing required field: email",
    "password is too short"
  ]
}
```

1.5 HTTP Status Codes

Code	Meaning	Typical Use
200	OK	Successful read or update
201	Created	Successful resource creation
204	No Content	Successful deletion
400	Bad Request	Invalid update payload
401	Unauthorized	Missing or invalid authentication token
403	Forbidden	User attempted to access another user's protected data
404	Not Found	Resource does not exist
409	Conflict	Duplicate username, email, or duplicate list-book relationship
422	Unprocessable Entity	Validation error in request body/query
502	Bad Gateway	External Open Library request failed

1.6 Ownership Rules

The system enforces ownership for protected data:

- users can only create reading lists for their own account
- users can only view their own reading lists
- users can only modify their own reading lists
- users can only create reviews for their own account
- users can only update or delete their own reviews

2. Authentication

2.1 Register User

Endpoint

```
POST /auth/register
```

Description

Create a new user account.

Request Body

```
{
  "username": "alice",
  "email": "alice@example.com",
  "password": "secret123"
}
```

Successful Response

```
{
  "id": 1,
  "username": "alice",
  "email": "alice@example.com",
  "created_at": "2026-04-20T05:00:00Z"
}
```

Possible Errors

- 409: username or email already exists
- 422: invalid email format or malformed request body

2.2 Login

Endpoint

```
POST /auth/login
```

Description

Authenticate a user and return a bearer token.

Request Body

```
{
  "email": "alice@example.com",
  "password": "secret123"
}
```

Successful Response

```
{
  "access_token": "token_here",
  "token_type": "bearer",
  "user": {
    "id": 1,
    "username": "alice",
    "email": "alice@example.com",
    "created_at": "2026-04-20T05:00:00Z"
  }
}
```

Possible Errors

- 401: invalid email or password
- 422: malformed request body

2.3 Current User

Endpoint

```
GET /auth/me
```

Description

Return the currently authenticated user.

Required Header

```
Authorization: Bearer <token>
```

Successful Response

```
{
  "id": 1,
  "username": "alice",
  "email": "alice@example.com",
  "created_at": "2026-04-20T05:00:00Z"
}
```

Possible Errors

- 401: authentication required
- 401: invalid authentication token

3. System Health

3.1 Health Check

Endpoint

```
GET /health
```

Description

Check whether the API server is running correctly.

Authentication Required

No

Successful Response

```
{
  "status": "ok"
}
```

4. Book Discovery

4.1 Search Books

Endpoint

```
GET /books/search
```

Description

Search Open Library works by keyword.

Query Parameters

Parameter	Type	Default	Description

Parameter	Type	Default	Description
q	string	required	Search keyword
limit	integer	10	Maximum number of returned results, range 1-50

Example Request

```
GET /books/search?q=python&limit=5
```

Successful Response

```
{
  "query": "python",
  "limit": 5,
  "results": [
    {
      "openlibrary_key": "/works/OL45804W",
      "title": "Learning Python",
      "author_name": "Mark Lutz",
      "first_publish_year": 1999,
      "cover_url": "https://covers.openlibrary.org/b/id/12345-L.jpg"
    }
  ]
}
```

Possible Errors

- 422: missing query or invalid limit
- 502: Open Library search request failed
- 502: unable to reach Open Library

4.2 Get Book Detail

Endpoint

```
GET /books/{openlibrary_key}
```

Description

Fetch a book detail record from Open Library.

Accepted Path Formats

- OL45804W
- /works/OL45804W

Successful Response

```
{
  "openlibrary_key": "/works/OL45804W",
  "title": "Learning Python",
  "author_name": null,
  "first_publish_year": null,
  "subject": "Python, Programming, Software Development",
  "cover_url": "https://covers.openlibrary.org/b/id/98765-L.jpg",
  "description": "A practical guide to Python."
}
```

Possible Errors

- 404: book not found in Open Library
- 502: Open Library detail request failed
- 502: unable to reach Open Library

5. Reading Lists

5.1 Create Reading List

Endpoint

```
POST /lists
```

Description

Create a new reading list for the authenticated user.

Authentication Required

Yes

Request Body

```
{
  "user_id": 1,
  "name": "Spring Reads",
  "description": "Books for the next semester break"
}
```

Successful Response

```
{
  "id": 1,
  "user_id": 1,
  "name": "Spring Reads",
  "description": "Books for the next semester break",
  "created_at": "2026-04-20T05:00:00Z",
  "updated_at": "2026-04-20T05:00:00Z"
}
```

Possible Errors

- 401: authentication required
- 403: user_id does not match authenticated user
- 422: malformed request body

5.2 List Reading Lists

Endpoint

```
GET /lists
```

Description

Return all reading lists owned by the authenticated user.

Authentication Required

Yes

Query Parameters

Parameter	Type	Default	Description
user_id	integer	current user	Optional filter, but must still match the authenticated user

Successful Response

```
[
  {
```

```
    "id": 1,
    "user_id": 1,
    "name": "Spring Reads",
    "description": "Books for the next semester break",
    "created_at": "2026-04-20T05:00:00Z",
    "updated_at": "2026-04-20T05:00:00Z"
  }
]
```

Possible Errors

- 401: authentication required
- 403: attempted to view another user's lists

5.3 Get Single Reading List

Endpoint

```
GET /lists/{list_id}
```

Description

Return one reading list and all its saved items.

Authentication Required

Yes

Successful Response

```
{
  "id": 1,
  "user_id": 1,
  "name": "Spring Reads",
  "description": "Books for the next semester break",
  "created_at": "2026-04-20T05:00:00Z",
  "updated_at": "2026-04-20T05:00:00Z",
  "items": [
    {
      "id": 1,
      "book_id": 4,
      "status": "finished",
      "added_at": "2026-04-20T05:10:00Z"
    }
  ]
}
```

Possible Errors

- 401: authentication required
- 403: attempted to view another user's list
- 404: reading list not found

5.4 Update Reading List

Endpoint

```
PUT /lists/{list_id}
```

Description

Update reading list name or description.

Authentication Required

Yes

Path Parameters

Parameter	Type	Description
list_id	integer	ID of the reading list to update

Request Body

```
{
  "name": "Updated List Name",
  "description": "Updated description"
}
```

Successful Response

```
{
  "id": 1,
  "user_id": 1,
  "name": "Updated List Name",
  "description": "Updated description",
  "created_at": "2026-04-20T05:00:00Z",
  "updated_at": "2026-04-20T05:20:00Z"
}
```

Possible Errors

- 400: no fields provided for update
- 401: authentication required
- 403: attempted to update another user's list
- 404: reading list not found

5.5 Delete Reading List

Endpoint

```
DELETE /lists/{list_id}
```

Description

Delete a reading list and its associated list items.

Authentication Required

Yes

Path Parameters

Parameter	Type	Description
list_id	integer	ID of the reading list to delete

Successful Response

No response body (`204 No Content`)

Possible Errors

- 401: authentication required
- 403: attempted to delete another user's list
- 404: reading list not found

5.6 Add Book to Reading List

Endpoint

```
POST /lists/{list_id}/items
```

Description

Add a book to a reading list. If the book does not already exist in the local database, it is created in the local `books` table first.

Authentication Required

Yes

Request Body

```
{
  "openlibrary_key": "/works/OL45804W",
  "title": "Learning Python",
  "author_name": "Mark Lutz",
  "first_publish_year": 1999,
  "subject": "Python, Programming",
  "cover_url": "https://covers.openlibrary.org/b/id/12345-L.jpg",
  "status": "to_read"
}
```

Successful Response

```
{
  "id": 1,
  "book_id": 4,
  "status": "to_read",
  "added_at": "2026-04-20T05:10:00Z"
}
```

Possible Errors

- 401: authentication required
- 403: attempted to modify another user's list
- 404: reading list not found
- 409: book already exists in this reading list

5.7 Update Reading List Item

Endpoint

```
PUT /lists/{list_id}/items/{item_id}
```

Description

Update the reading status of one list item.

Authentication Required

Yes

Path Parameters

Parameter	Type	Description
list_id	integer	ID of the reading list that owns the item
item_id	integer	ID of the reading list item to update

Request Body

```
{
  "status": "finished"
}
```

Successful Response

```
{
  "id": 1,
  "book_id": 4,
  "status": "finished",
  "added_at": "2026-04-20T05:10:00Z"
}
```

Possible Errors

- 401: authentication required
- 403: attempted to modify another user's list
- 404: reading list not found
- 404: reading list item not found

5.8 Delete Reading List Item

Endpoint

```
DELETE /lists/{list_id}/items/{item_id}
```

Description

Remove a book entry from a reading list.

Authentication Required

Yes

Path Parameters

Parameter	Type	Description
list_id	integer	ID of the reading list that owns the item
item_id	integer	ID of the reading list item to delete

Successful Response

No response body (`204 No Content`)

Possible Errors

- 401: authentication required
- 403: attempted to modify another user's list
- 404: reading list not found
- 404: reading list item not found

6. Reviews

6.1 Create Review

Endpoint

```
POST /reviews
```

Description

Create a review for a locally cached book.

Authentication Required

Yes

Request Body

```
{
  "user_id": 1,
  "book_id": 2,
  "rating": 5,
  "review_text": "Helpful and well structured."
}
```

Successful Response

```
{
  "id": 1,
  "user_id": 1,
  "book_id": 2,
  "rating": 5,
  "review_text": "Helpful and well structured.",
  "created_at": "2026-04-20T05:15:00Z",
  "updated_at": "2026-04-20T05:15:00Z"
}
```

Possible Errors

- 401: authentication required
- 403: attempted to create review for another user id
- 404: book not found
- 422: invalid rating or malformed request body

6.2 Get Review by ID

Endpoint

```
GET /reviews/{review_id}
```

Description

Return one review by id.

Path Parameters

Parameter	Type	Description
review_id	integer	ID of the review to retrieve

Successful Response

```
{
  "id": 1,
  "user_id": 1,
  "book_id": 2,
  "rating": 5,
  "review_text": "Helpful and well structured.",
  "created_at": "2026-04-20T05:15:00Z",
  "updated_at": "2026-04-20T05:15:00Z"
}
```

Possible Errors

- 404: review not found

6.3 List Reviews for One Book

Endpoint

```
GET /reviews/book/{book_id}
```

Description

Return all locally stored reviews for a single local book id.

Path Parameters

Parameter	Type	Description
book_id	integer	Local database ID of the cached book

Successful Response

```
[
  {
    "id": 1,
    "user_id": 1,
    "book_id": 2,
    "rating": 5,
    "review_text": "Helpful and well structured.",
    "created_at": "2026-04-20T05:15:00Z",
    "updated_at": "2026-04-20T05:15:00Z"
  }
]
```

Possible Errors

- 404: book not found

6.4 Update Review

Endpoint

```
PUT /reviews/{review_id}
```

Description

Update the rating and/or review text for one review.

Authentication Required

Yes

Path Parameters

Parameter	Type	Description
review_id	integer	ID of the review to update

Request Body

```
{
  "rating": 4,
  "review_text": "Still good, but slightly repetitive."
}
```

Successful Response

```
{
  "id": 1,
  "user_id": 1,
  "book_id": 2,
```

```
  "rating": 4,
  "review_text": "Still good, but slightly repetitive.",
  "created_at": "2026-04-20T05:15:00Z",
  "updated_at": "2026-04-20T05:25:00Z"
}
```

Possible Errors

- 400: no fields provided for update
- 401: authentication required
- 403: attempted to update another user's review
- 404: review not found
- 422: invalid rating value

6.5 Delete Review

Endpoint

```
DELETE /reviews/{review_id}
```

Description

Delete one review.

Authentication Required

Yes

Path Parameters

Parameter	Type	Description
review_id	integer	ID of the review to delete

Successful Response

No response body (`204 No Content`)

Possible Errors

- 401: authentication required
- 403: attempted to delete another user's review
- 404: review not found

7. Analytics and Recommendations

7.1 Genre Analytics

Endpoint

```
GET /analytics/genres
```

Description

Return the most common subjects across locally cached books.

Successful Response

```
{
  "genres": [
    {
      "name": "Python",
      "count": 4
```

```
    },
    {
      "name": "Programming",
      "count": 3
    }
  ]
}
```

7.2 User Preference Analytics

Endpoint

```
GET /analytics/user/{user_id}/preferences
```

Description

Return a user's top subjects, top authors, and average rating based on locally saved books and reviews.

Path Parameters

Parameter	Type	Description
user_id	integer	ID of the user whose preference profile will be calculated

Successful Response

```
{
  "user_id": 1,
  "favorite_subjects": ["Python", "Programming"],
  "favorite_authors": ["Mark Lutz"],
  "average_rating": 4.5
}
```

Possible Errors

- 404: user not found

7.3 Recommendations

Endpoint

```
GET /analytics/recommendations/user/{user_id}
```

Description

Return rule-based recommendations generated from the user's saved books and reviews.

Path Parameters

Parameter	Type	Description
user_id	integer	ID of the user for whom recommendations are generated

Current Recommendation Strategy

- increases subject weight from saved books
- increases author and subject weight from highly rated books
- excludes books already saved or reviewed by the user
- returns a short reason with each result

Successful Response

```
{
  "user_id": 1,
  "recommendations": [
    {
```

```
      "book_id": 7,
      "openlibrary_key": "/works/OL12345W",
      "title": "Advanced Python Systems",
      "author_name": "Chris Builder",
      "subject": "Python, Software Engineering",
      "cover_url": "https://example.com/cover.jpg",
      "reason": "matches your preferred subjects: Python; includes an author you rate highl
y: Chris Builder"
    }
  ]
}
```

Possible Errors

- 404: user not found

8. Endpoint Summary

Category	Method	Endpoint	Auth Required	Purpose
System	GET	/health	No	Health check
Authentication	POST	/auth/register	No	Create user
Authentication	POST	/auth/login	No	Login and get token
Authentication	GET	/auth/me	Yes	Get current user
Books	GET	/books/search	No	Search Open Library books
Books	GET	/books/{openlibrary_key}	No	Get Open Library book detail
Lists	POST	/lists	Yes	Create reading list
Lists	GET	/lists	Yes	List reading lists
Lists	GET	/lists/{list_id}	Yes	Get one reading list
Lists	PUT	/lists/{list_id}	Yes	Update reading list
Lists	DELETE	/lists/{list_id}	Yes	Delete reading list
List Items	POST	/lists/{list_id}/items	Yes	Add book to list
List Items	PUT	/lists/{list_id}/items/{item_id}	Yes	Update list item status
List Items	DELETE	/lists/{list_id}/items/{item_id}	Yes	Delete list item
Reviews	POST	/reviews	Yes	Create review
Reviews	GET	/reviews/{review_id}	No	Get one review
Reviews	GET	/reviews/book/{book_id}	No	Get all reviews for one book
Reviews	PUT	/reviews/{review_id}	Yes	Update review
Reviews	DELETE	/reviews/{review_id}	Yes	Delete review
Analytics	GET	/analytics/genres	No	Genre frequency summary
Analytics	GET	/analytics/user/{user_id}/preferences	No	User preferences
Analytics	GET	/analytics/recommendations/user/{user_id}	No	Recommendations

Notes

- Book search and detail endpoints depend on Open Library availability.

- Reading lists, reviews, analytics, and recommendations depend on local SQL data.
- Interactive generated API docs remain available through Swagger UI and ReDoc.